FST BeniMellal

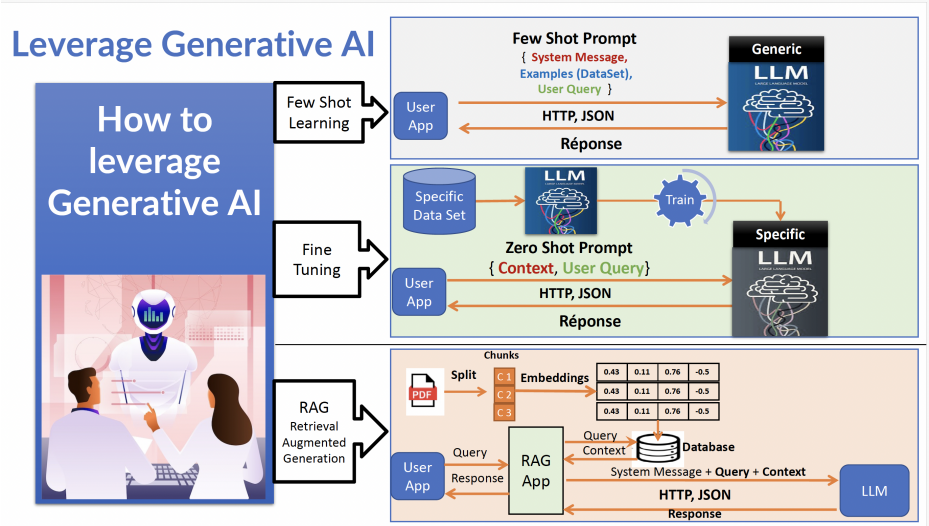
Master : IA (S3)

Systèmes Distribués Agentiques

RAG : Retrieval-Augmented Generation

Architecture et Pipeline Complet

Semestre 3 – Année Universitaire 2026-2027


Leverage Generative AI
How to leverage Generative AI
Few Shot Learning
Fine Tuning
RAG
Retrieval Augmented Generation
Few Shot Prompt
{ System Message, Examples (DataSet), User Query }
User App
HTTP, JSON
Réponse
Generic
LLM
Specific Data Set
Train
Specific
Zero Shot Prompt
{ Context, User Query}
Chunks
Split
Embeddings
C 1
C 2
C 3
0.43 0.11 0.76 -0.5
Query
Context
Database
Query
Response
RAG App
System Message + Query + Context
HTTP, JSON
Response

Plan de la séance

Introduction au RAG

Phase d'Indexation

Phase de Recherche (Retrieval)

Phase de Génération

Les limites des LLMs

- **Hallucinations**
  - Génèrent des informations fausses avec confiance
  - Inventent des citations et des faits
- **Données obsolètes**
  - Connaissances figées à la date d'entraînement
  - Aucune mise à jour en temps réel
- **Pas de données privées**
  - Aucun accès aux documents internes
  - Personnalisation limitée sans intervention
- **Fenêtre de contexte limitée**
  - Nombre de tokens en entrée limité
  - Impossible de traiter des corpus entiers

Important

Ces limites rendent les LLMs peu fiables pour des applications critiques sans mécanisme de vérification.

Exemple concret d'hallucination

Exemple

Question : « Quel est le PIB du Maroc en 2024 ? »

Réponse du LLM (sans RAG) :
« Le PIB du Maroc en 2024 est de 142 milliards de dollars. »

Important

Ce chiffre peut être inventé ou obsolète.

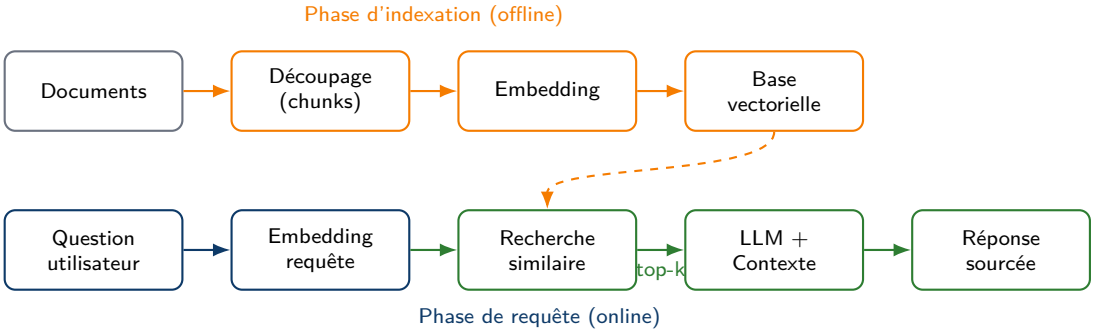
Important

Le LLM ne sait pas qu'il ne sait pas. Il génère une réponse plausible mais potentiellement fausse. C'est le problème fondamental des hallucinations.

Définition du RAG

Définition

Le RAG (*Retrieval-Augmented Generation*) est une technique qui combine la recherche d'information dans une base de connaissances avec la génération de texte par un LLM, afin de produire des réponses fondées sur des données factuelles.

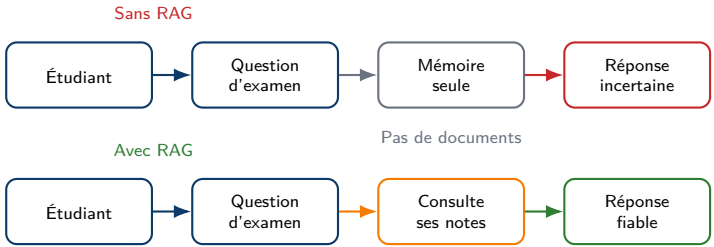


Analogie pédagogique

Astuce

Le RAG est comme un étudiant qui consulte ses notes avant de répondre à une question d'examen.

- **Sans RAG :** l'étudiant répond de mémoire → risque d'erreurs
- **Avec RAG :** l'étudiant consulte ses documents → réponse fiable et sourcée



Retrieval = consultation des notes | LLM = étudiant | base de connaissances = documents

3 approches pour adapter un LLM

Prompt Engineering

Rédiger des instructions précises (prompts) pour guider le LLM, sans modifier le modèle ni ajouter de données externes.

Ex. : « Réponds en 3 points, en français. »

Fine-tuning

Ré-entraîner le LLM sur un jeu de données spécifique pour qu'il apprenne un domaine ou un style. Cela modifie les poids du modèle.

Ex. : entraîner GPT sur des rapports médicaux.

RAG

Rechercher des documents pertinents dans une base de connaissances, puis les injecter dans le prompt pour que le LLM s'appuie dessus.

Ex. : chatbot FAQ d'entreprise.

Important

Ces 3 approches ne sont pas exclusives : on peut combiner le prompt engineering avec le RAG, ou utiliser le fine-tuning en complément du RAG pour améliorer le style des réponses.

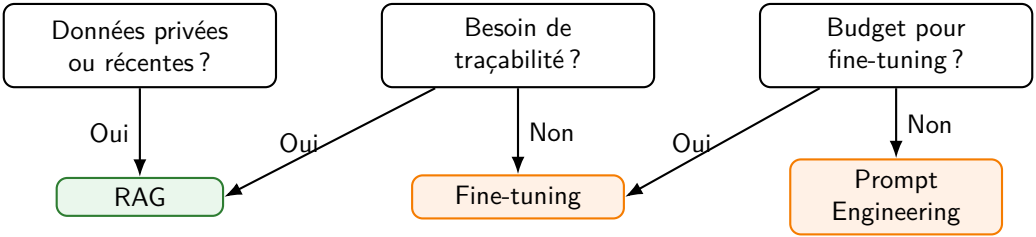
RAG vs Fine-tuning vs Prompt Engineering

Critère	Prompt Engineering	Fine-tuning	RAG
Coût	✓Faible	✗Élevé	★Modéré
Effort	✓Minimal	✗Important	★Modéré
Fraîcheur données	✗Statique	✗Statique	✓Dynamique
Précision domaine	✗Limitée	✓Bonne	✓Très bonne
Données privées	✗Non	★Partiel	✓Oui
Traçabilité	✗Aucune	✗Faible	✓Sources citées
Mise à jour	✓Immédiate	✗Ré-entraînement	✓Temps réel

Astuce

Le RAG offre le meilleur compromis entre coût, fraîcheur et précision.

Quand utiliser le RAG ?



Cas d'usage concrets du RAG

- **Support client :** chatbot qui consulte la base FAQ et la documentation produit pour répondre aux clients.
- **Recherche juridique :** assistant qui recherche dans les textes de loi et la jurisprudence pour aider les avocats.
- **Domaine médical :** outil d'aide au diagnostic basé sur la littérature médicale et les protocoles cliniques.
- **Éducation :** tuteur intelligent qui puise dans les cours et manuels pour accompagner les étudiants.

Le Pipeline RAG complet

1. Indexation

- Chargement docs
- Découpage (chunking)
- Embeddings
- Stockage vectoriel

2. Recherche

- Embedding requête
- Similarité vectorielle
- Top-K résultats
- Re-ranking

3. Génération

- Construction prompt
- Injection contexte
- Appel LLM
- Réponse sourcée

Pipeline RAG – Phase 1 : Indexation

Chargement docs

Lecture des documents bruts depuis leurs sources (PDF, pages web, bases de données). Utilise des loaders spécialisés par format.

Découpage (chunking)

Division des documents en morceaux de taille fixe (ex. : 500 tokens) avec chevauchement, pour que chaque chunk soit indexable séparément.

Embeddings

Transformation de chaque chunk en un vecteur numérique (ex. : 768 ou 1536 dimensions) via un modèle d'embedding. Cela capture le sens sémantique du texte.

Stockage vectoriel

Sauvegarde des vecteurs dans une base vectorielle (FAISS, Qdrant, Pinecone) avec les métadonnées associées, prêts pour la recherche.

Pipeline RAG – Phase 2 : Recherche

Embedding requête

La question de l'utilisateur est convertie en vecteur avec le même modèle d'embedding utilisé pour l'indexation, afin de permettre la comparaison.

Similarité vectorielle

Calcul de la distance cosinus entre le vecteur de la requête et tous les vecteurs stockés. Les chunks les plus proches sémantiquement obtiennent les meilleurs scores.

Top-K résultats

Sélection des K documents les plus proches (ex. : K=5). Seuls les chunks ayant le score de similarité le plus élevé sont retenus.

Re-ranking

La recherche vectorielle est rapide mais approximative. Le re-ranking reclasse les Top-K avec un modèle plus précis (cross-encoder) qui lit la requête et le chunk ensemble pour juger leur pertinence réelle.

Pipeline RAG – Phase 3 : Génération

Construction prompt

Assemblage du template de prompt : instructions système, format de réponse attendu, et consignes (ex. : « réponds uniquement à partir du contexte fourni »).

Appel LLM

Envoi du prompt complet (instructions + contexte + question) au modèle de langage (GPT-4, Claude, Llama) qui génère la réponse.

Injection contexte

Insertion des documents récupérés (après re-ranking) dans le prompt, avec leurs métadonnées (source, page), pour que le LLM s'appuie dessus.

Réponse sourcée

Réponse finale fondée sur les documents sources. Traçable et vérifiable : on peut remonter aux passages exacts utilisés, réduisant ainsi les hallucinations.

Vue d'ensemble des 3 phases

	1. Indexation	2. Recherche	3. Génération
Input	Documents bruts (PDF, HTML, etc.)	Question utilisateur	Question + contexte récupéré
Output	Base vectorielle indexée	Top-K documents pertinents	Réponse en langage naturel
Outils	LangChain, LlamaIndex, Unstructured	FAISS, Qdrant, Pinecone	GPT-4, Claude, Llama
Fréquence	Hors-ligne (batch)	Temps réel (par requête)	Temps réel (par requête)

Astuce

La phase d'indexation est réalisée une seule fois (ou périodiquement), tandis que les phases de recherche et génération sont exécutées à chaque requête.

Évolution historique du RAG



Définition

Le concept de RAG a été formalisé par Lewis et al. (2020) chez Facebook AI Research. Depuis, il est devenu le *pattern* dominant pour connecter les LLMs à des données externes.

Pourquoi le RAG est-il si populaire ?

- ✓ Pas de ré-entraînement : on garde le LLM tel quel
- ✓ Données à jour : la base de connaissances est mise à jour indépendamment
- ✓ Traçabilité : chaque réponse cite ses sources
- ✓ Sécurité : les données privées restent dans l'infrastructure
- ✓ Coût réduit : pas de GPU pour le fine-tuning
- ✓ Modularité : on peut changer le LLM, la base vectorielle ou les documents

Important

Le RAG n'est pas une solution magique. Sa qualité dépend directement de la qualité des documents indexés et de la pertinence de la recherche.

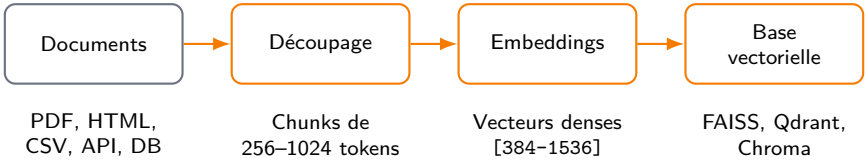
Phase d'Indexation

Transformer des documents bruts en vecteurs recherchables

Vue d'ensemble de la phase d'indexation

Définition

La phase d'indexation transforme des documents bruts en représentations vectorielles stockées dans une base de données optimisée pour la recherche par similarité.



Étape 1 – Chargement des documents

- **PDF :** rapports, articles, manuels
- **HTML :** pages web, documentation
- **CSV/Excel :** données tabulaires
- **API :** services web, REST
- **Base de données :** SQL, NoSQL
- **Texte :** Markdown, TXT, DOCX

Astuce

Des bibliothèques comme LangChain, LlamaIndex et Unstructured fournissent des loaders prêts à l'emploi pour chaque format.

Extraction et nettoyage des données

Important

La qualité du RAG dépend directement de la qualité des données en entrée. *Garbage in, garbage out* : si les documents indexés sont mal extraits ou bruités, le RAG produira des réponses de mauvaise qualité, quel que soit le LLM utilisé.

Étapes de prétraitement :

1. Extraction du texte : OCR pour les images, parsing pour les PDF
2. Nettoyage : suppression des en-têtes/pieds de page, caractères spéciaux
3. Normalisation : encodage UTF-8, uniformisation des formats
4. Enrichissement : ajout de métadonnées (source, date, auteur)

Exemple

Un PDF de 100 pages $\rightarrow$ extraction texte $\rightarrow$ nettoyage $\rightarrow$ 85 pages de contenu utile avec métadonnées (titre, chapitre, page).

Étape 2 – Découpage (Chunking)

Définition

Le *chunking* consiste à découper les documents en morceaux (chunks) de taille appropriée pour l'indexation et la recherche. C'est une étape critique du pipeline RAG.

Pourquoi découper ?

- Les modèles d'embeddings ont une limite de tokens
- Des chunks trop grands $\rightarrow$ embedding dilué, recherche imprécise
- Des chunks trop petits $\rightarrow$ perte de contexte, résultats fragmentés
- Objectif : trouver le bon équilibre entre précision et contexte

Stratégies de chunking

Taille fixe

Chunk 1 (256 tokens)
Chunk 2 (256 tokens)
Chunk 3 (256 tokens)

✓ Simple et rapide
✗ Coupe en plein milieu

Par phrases

Phrase 1
Phrases 2–3
Phrase 4

✓ Respecte la syntaxe
✗ Chunks trop courts

Récursif

Section complète
Paragraphe
Sous-section

✓ Cohérent sémantiquement
✗ Plus complexe

Sémantique

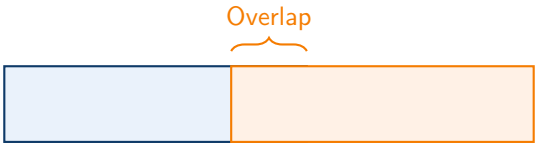
Idée complète A
Idée B
Idée complète C

✓ Meilleure qualité
✗ Coûteux (appel LLM)

Notion d'overlap (chevauchement)

Définition

L'*overlap* est le nombre de tokens partagés entre deux chunks consécutifs. Il permet de préserver le contexte aux frontières.



Astuce

Valeur typique : `chunk_size=512`, `chunk_overlap=50-100`. L'overlap représente généralement 10–20% de la taille du chunk.

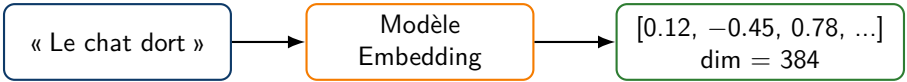
Comparaison des stratégies de chunking

Stratégie	**chunk_size**	**overlap**	**Avantages**	**Inconvénients**
Taille fixe	256–512	50–100	Simple, rapide, prévisible	Coupe le contexte
Par phrases	Variable	1–2 phrases	Respecte la syntaxe	Chunks souvent trop courts
Récursif	512–1024	50–200	Cohérence sémantique	Plus complexe
Sémantique	Variable	Auto	Meilleure qualité	Coûteux (appel LLM)

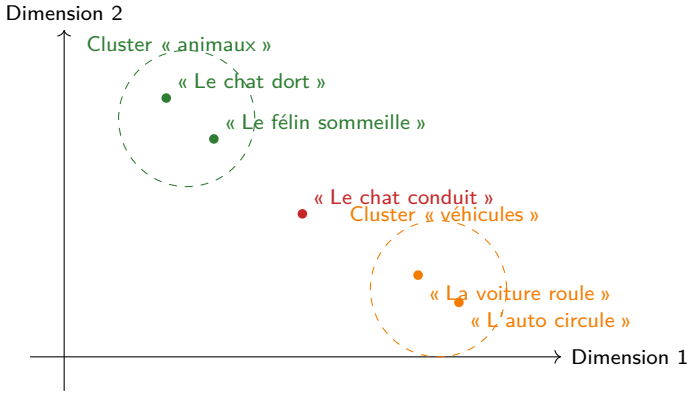
Étape 3 – Génération d'embeddings

Définition

Un embedding est une représentation vectorielle dense d'un texte dans un espace à n dimensions, où les textes sémantiquement proches ont des vecteurs proches.



Comment fonctionnent les embeddings ?



Astuce

Les embeddings capturent le sens sémantique du texte. Deux phrases qui disent la même chose auront des vecteurs très proches, même avec des mots différents.

Modèles d'embeddings populaires

Modèle	Fournisseur	Dim.	Multilingue	Coût
all-MiniLM-L6-v2	HuggingFace	384	✗Non	✓Gratuit
multilingual-e5-large	HuggingFace	1024	✓Oui	✓Gratuit
text-embedding-3-small	OpenAI	1536	✓Oui	★Payant
embed-multilingual-v3	Cohere	1024	✓Oui	★Payant
voyage-3	Voyage AI	1024	✓Oui	★Payant

Astuce

Pour un projet en français, privilégiez un modèle multilingue. Les modèles open-source de HuggingFace (`sentence-transformers`) sont idéaux pour débuter sans coût.

Étape 4 – Stockage vectoriel

Bases vectorielles populaires :

- Qdrant : riche en fonctionnalités, open-source
- FAISS (Meta) : rapide, en mémoire
- Chroma : simple, léger, local
- Pinecone : cloud managé
- Weaviate : open-source, hybride

Critères de choix :

- Volume : millions de vecteurs ?
- Latence : temps réel requis ?
- Filtrage : métadonnées ?
- Hébergement : cloud vs local ?
- Coût : budget disponible ?

Important

Le choix de la base vectorielle sera détaillé en Partie 2. Pour commencer, Qdrant ou FAISS sont parfaits pour le prototypage.

Pipeline d'indexation complet

Astuce

Résumé du pipeline d'indexation :

1. Charger les documents depuis les sources
2. Nettoyer et normaliser le contenu
3. Découper en chunks (`RecursiveCharacterTextSplitter`)
4. Générer les embeddings (`sentence-transformers`)
5. Stocker dans la base vectorielle (Qdrant / FAISS)
6. Enrichir avec des métadonnées (source, date, etc.)

Exemple

1000 pages PDF $\rightarrow$ 5000 chunks de 512 tokens $\rightarrow$ 5000 vecteurs de 384 dimensions $\rightarrow$ indexés dans Qdrant en $\sim$30 secondes.

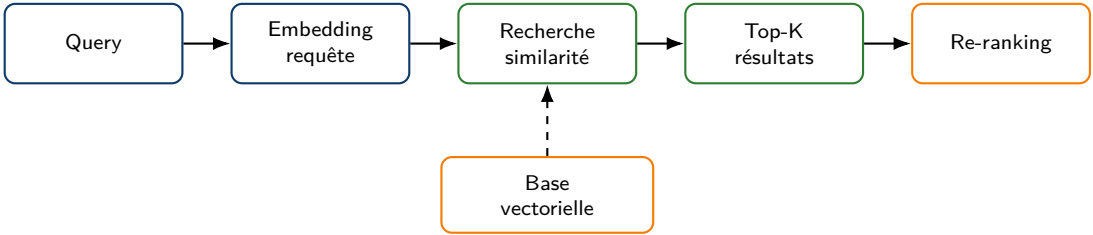
Phase de Recherche (Retrieval)

Trouver les documents les plus pertinents pour une requête

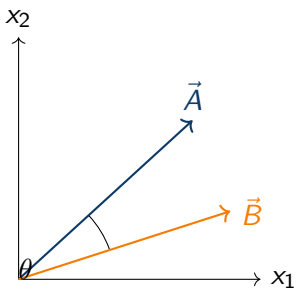
Vue d'ensemble de la phase de recherche

Définition

La phase de retrieval transforme la question en vecteur, puis recherche les K documents les plus similaires dans la base vectorielle.



Métriques de similarité vectorielle



Similarité cosinus :

$$\cos(\theta) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \times \|\vec{B}\|}$$

- ✓ Valeur entre -1 et 1
- ✓ Indépendant de la norme
- Métrique la plus utilisée en RAG

Comparaison des métriques de similarité

Métrique	Formule	Usage	Plage
Cosinus	$\frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \cdot \|\vec{B}\|}$	Similarité sémantique, RAG	$[-1, 1]$
Euclidienne	$\sqrt{\sum (A_i - B_i)^2}$	Clustering, k-NN	$[0, +\infty[$
Produit scalaire	$\vec{A} \cdot \vec{B} = \sum A_i B_i$	Vecteurs normalisés	$]-\infty, +\infty[$

Astuce

La similarité cosinus est le choix standard en RAG car elle mesure l'angle entre les vecteurs, indépendamment de leur longueur. Deux documents sur le même sujet auront un cosinus proche de 1.

Recherche Top-K

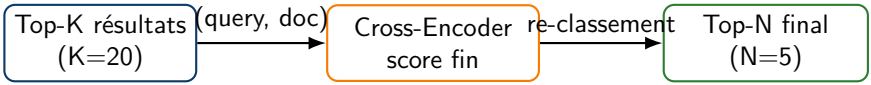
Résultats classés par similarité (K=5) :

- ✓ Doc 3 – Score : 0.95
- ✓ Doc 7 – Score : 0.89
- ★ Doc 1 – Score : 0.82
- ★ Doc 12 – Score : 0.71
- ✗ Doc 5 – Score : 0.54

Le choix de K est crucial :

- ✗ **K trop petit** (ex. : K=1) : on risque de manquer l'information pertinente
- ✗ **K trop grand** (ex. : K=20) : on injecte du bruit dans le prompt
- ✓ **K=3 à 5** : bon compromis entre couverture et précision

Re-ranking avec Cross-Encoder



Définition

Un Cross-Encoder lit la requête et le document ensemble pour juger leur pertinence mutuelle. Contrairement à la recherche vectorielle qui compare des vecteurs calculés séparément, il comprend les relations entre la question et le document.

Astuce

Stratégie en 2 étapes : (1) récupérer K=20 candidats rapidement avec la similarité vectorielle, (2) reclasser avec un cross-encoder pour garder les N=5 meilleurs.

Recherche hybride : deux façons de chercher

Définition

Il existe deux façons de chercher un document pertinent. La recherche hybride les combine pour obtenir le meilleur des deux.

Recherche par sens (Dense)

Cherche les documents qui parlent de la même chose, même avec des mots différents.

Requête : « animal domestique »

- ✓ Trouve : « Le chat est un compagnon fidèle »
- ✓ Trouve : « Les félins aiment dormir »
- ✗ Peut manquer un document contenant exactement « animal domestique »

Recherche par mots-clés (Sparse)

Cherche les documents qui contiennent exactement les mêmes mots.

Requête : « animal domestique »

- ✓ Trouve : « L'animal domestique le plus populaire... »
- ✗ Rate : « Le chat est un compagnon fidèle »
- ✗ Ne comprend pas les synonymes

Astuce

Recherche hybride = combiner les deux : score final $= \alpha \times$ score dense $+ (1 - \alpha) \times$ score lexical. Typiquement, $\alpha = 0.5$.

Filtrage par métadonnées

Définition

Le filtrage par métadonnées permet de restreindre la recherche vectorielle à un sous-ensemble de documents, en utilisant des attributs comme la source, la date, la catégorie, etc.

Exemple

Requête : « Politique de remboursement »
Filtre : `source = "FAQ" ET date > "2024-01-01"`
$\rightarrow$ ne recherche que dans les documents FAQ récents, pas dans tout le corpus.

Types de filtres courants :

- Égalité : `categorie == "juridique"`
- Plage : `date >= "2024-01-01"`
- Liste : `source IN ["FAQ", "Manuel"]`

Évaluation de la qualité du retrieval

Métrique	Question à laquelle elle répond	Plage
Recall@K	Parmi tous les documents pertinents, combien sont dans le Top-K ?	[0,1]
Precision@K	Parmi les K documents retournés, combien sont vraiment pertinents ?	[0,1]

Exemple

4 documents pertinents existent dans la base. Le Top-5 retourne 3 d'entre eux + 2 non pertinents.
✓ Recall@5 $= 3/4 = 0.75$
✓ Precision@5 $= 3/5 = 0.60$

Important

Objectif : un bon retrieval vise un Recall@5 ≥ 0.8. Si le retrieval rate un document clé, le LLM n'a aucun moyen de le deviner.

- ✗ Mauvais retrieval + bon LLM = mauvaise réponse
- ✓ Bon retrieval + bon LLM = bonne réponse

Erreurs courantes en retrieval

Important

Pièges à éviter lors de la phase de recherche :

- Chunks trop grands → le vecteur mélange trop de sujets, résultats imprécis
- Chunks trop petits → perte de contexte, réponses incomplètes
- K trop faible → manquer l'information pertinente
- K trop élevé → bruit, contexte trop long pour le LLM
- Pas de re-ranking → résultats de qualité moyenne
- Ignorer les métadonnées → recherche dans tout le corpus inutilement
- Mauvais modèle d'embedding → représentations sémantiques pauvres

Phase de Génération

Produire une réponse fidèle à partir du contexte récupéré

Injection de contexte dans le prompt



Définition

La génération augmentée consiste à injecter les documents récupérés dans le prompt du LLM, en lui demandant de répondre uniquement sur la base de ce contexte.

Template de prompt RAG

```
SYSTEM_PROMPT = """Tu es un assistant qui repond
aux questions en utilisant UNIQUEMENT le contexte
fourni. Si tu ne trouves pas la reponse dans le
contexte, dis : "Je ne dispose pas de cette
information dans mes sources."

Contexte :
{context}

Cite toujours tes sources entre crochets [Source X].
"""

USER_PROMPT = """Question : {question}

Reponds de maniere precise et concise en te basant
uniquement sur le contexte fourni."""
```

Astuce

Le prompt doit explicitement demander au LLM de : (1) se limiter au contexte, (2) citer ses sources, (3) avouer quand il ne sait pas.

Paramètres de génération

Paramètre	Ce qu'il contrôle	Valeur RAG
temperature	Créativité du LLM. À 0 : réponse toujours identique et fidèle. À 1 : réponse variée mais risque d'inventer.	0.0 – 0.3
max_tokens	Longueur maximale de la réponse générée.	500 – 2000
top_p	Filtre sur le vocabulaire. `top_p=0.9` signifie : ne considérer que les mots dont les probabilités cumulées atteignent 90%.	0.9

Important

Pour le RAG, utilisez une température basse (0.0–0.3) : on veut que le LLM restitue fidèlement le contenu des documents, pas qu'il improvise.

Synthèse de la réponse : bonnes pratiques

Astuce

Principes pour une génération de qualité :

- ✓ **Fidélité (Faithfulness)** : ne répondre qu'à partir du contexte fourni
- ✓ **Citation des sources** : indiquer d'où vient chaque information
- ✓ **Aveu d'ignorance** : dire « je ne sais pas » plutôt qu'inventer
- ✓ **Concision** : réponse directe et structurée
- ✓ **Cohérence** : synthétiser les informations de plusieurs chunks

Important

Ne jamais demander au LLM d'« utiliser ses connaissances » en plus du contexte. Cela réintroduit le risque d'hallucination.

Démonstration RAG étape par étape

1. **Question de l'utilisateur**

Exemple

« Quelle est la politique de retour de l'entreprise ? »

2. **Recherche dans la base vectorielle**
   La question est convertie en vecteur, puis comparée aux chunks indexés. Top-5 résultats récupérés.
3. **Re-ranking (cross-encoder)**
   Top-3 après re-ranking :
   - ✓ [Source 1] « Les retours sont acceptés sous 30 jours... » score : 0.97
   - ✓ [Source 2] « Le remboursement est effectué sous 5 jours... » score : 0.91
   - ★ [Source 3] « Les frais de retour sont à la charge du client... » score : 0.84
4. **Génération par le LLM**

Exemple

« Selon notre politique, les retours sont acceptés sous 30 jours [Source 1]. Le remboursement

Pipeline RAG complet – Résumé

